Name (Last, First): _____

These problems are about `42k_boot_to_life`, `43k_traps` and `44k_interrupts`. There are 3 problems on 6 pages. Answer each part in the space below it, and show your work.

1. **Boot order (42k).** In 42k you wrote `kinit`, which brings the kernel up one subsystem at a time, in this order:

```
unsafe fn kinit() {
    uart::init();                       // 1. the console, so we can print
    kalloc::init();                     // 2. the page allocator
    vm::kvminithart(vm::kvmmake());     // 3. build the kernel page table, then turn the MMU on
    proc::init();                       // 4. the process table
}
```

   These are the page-allocator functions (from 32k) and the start of `kvmmake` (from 39k):

```
static mut FREELIST: *mut Run = ptr::null_mut();   // the free list starts EMPTY
pub unsafe fn init() {                     // kalloc::init(): put every free page of RAM
    let start = &end as *const u8 as usize;   //   (from the end of the kernel up to
    free_range(start, PHYSTOP);               //   PHYSTOP) on the free list
}
unsafe fn free_range(start: usize, stop: usize) {
    let mut p = pgroundup(start);
    while p + PGSIZE <= stop { kfree(p as *mut u8); p += PGSIZE; }
}
pub unsafe fn kfree(pa: *mut u8) {         // push page pa onto the free list
    let r = pa as *mut Run;
    (*r).next = FREELIST;
    FREELIST = r;
}
pub unsafe fn kalloc() -> *mut u8 {        // pop a page off the free list (null if empty)
    let r = FREELIST;
    if !r.is_null() { FREELIST = (*r).next; }
    r as *mut u8
}
pub unsafe fn kvmmake() -> *mut Pte {
    let root = kalloc::kalloc() as *mut Pte;   // a page for the root page table
    if root.is_null() { return ptr::null_mut(); }
    // ... then more kalloc() pages for the lower-level tables, and finally:
    root
}
```

(a) A student writes `kinit` with the page table built *before* `kalloc::init()`:

```
uart::init();
vm::kvminithart(vm::kvmmake());
kalloc::init();
proc::init();
```

(i) What does `kvmmake()` return, and why? (ii) Fix the order, and say in one sentence why that step has to come first.

(b) Why does `uart::init()` come first? After booting, the kernel idles like this:

```
loop { unsafe { core::arch::asm!("wfi") }; }   // wfi: "wait for interrupt"
```

Why `wfi` inside the loop, instead of an empty `loop {}`?

2. **Traps (43k).** When the running code hits a breakpoint (`ebreak`, a 4-byte instruction), the hardware traps into the kernel. On a trap it does exactly three things: it writes *why* into `scause` (3 = breakpoint), writes the trapping instruction's address into `sepc`, and jumps to the address in `stvec`.

```
static mut TRAP_COUNT: usize = 0;          // breakpoints handled so far
pub unsafe fn init() {                     // called once, at boot
    let addr = vector_addr();              // the address of kernelvec
    asm!("csrw stvec, {}", in(reg) addr);  // traps now jump to kernelvec
}
kernelvec:                                 // (assembly) every trap lands here
    addi sp, sp, -128                      // save the 16 caller-saved registers
    sd ra, 0(sp)                           //   (ra, t0-t6, a0-a7) on the stack
    ...
    call kerneltrap                        // run the Rust handler below
    ld ra, 0(sp)                           // restore the same 16 registers
    ...
    addi sp, sp, 128
    sret                                   // return to the address in sepc

#[no_mangle]
pub extern "C" fn kerneltrap() {
    unsafe {
        let scause: usize;
        let sepc: usize;
        asm!("csrr {}, scause", out(reg) scause);   // why did we trap?
        asm!("csrr {}, sepc", out(reg) sepc);       // where did we trap?
        if scause == 3 {                            // a breakpoint:
            TRAP_COUNT += 1;                        //   count it
            asm!("csrw sepc, {}", in(reg) sepc + 4); //  resume after the ebreak
        }
    }
}
```

(a) After `init()`, an `ebreak` at address `0x8000_0568` runs. `kernelvec` is at `0x8000_0af0`. Fill in the table. The first row is done for you.

| | Value |
|---|---|
| `scause` when `kerneltrap` starts | 3 (a breakpoint) |
| `sepc` when `kerneltrap` starts | |
| address the CPU jumps to on the trap | |
| `sepc` when `kerneltrap` returns | |
| address where `sret` resumes | |

(b) A student's `kerneltrap` counts the breakpoint but forgets the `csrw sepc` line. What happens after `sret`, and why?

3. **Interrupts (44k).** 44k adds a timer that ticks about every 0.1 seconds. Each tick arrives through the same trap path (`kernelvec` → `kerneltrap`), and while it is waiting to be handled it is *pending*: bit 1 of the `sip` register (SSIP) is set. The CPU takes a pending, enabled interrupt as soon as it can, including right after `sret`.

```
static mut TICKS: usize = 0;                        // timer ticks handled so far
static mut TRAP_COUNT: usize = 0;                   // breakpoints handled so far
pub unsafe fn intr_on() {                           // let interrupts in
    asm!("csrs sie, {}", in(reg) 1usize << 1);      // SSIE: allow this interrupt source
    asm!("csrs sstatus, {}", in(reg) 1usize << 1);  // SIE: interrupts on, globally
}
pub extern "C" fn kerneltrap() {
    unsafe {
        let scause: usize; let sepc: usize;
        asm!("csrr {}, scause", out(reg) scause);
        asm!("csrr {}, sepc", out(reg) sepc);
        if (scause >> 63) == 1 {                    // top bit 1: an INTERRUPT
            if scause & 0xff == 1 {                 //   code 1: a timer tick
                let sip: usize;
                asm!("csrr {}, sip", out(reg) sip);
                asm!("csrw sip, {}", in(reg) sip & !2);  // clear SSIP (bit 1): handled
                TICKS += 1;
            }
        } else {                                    // top bit 0: an EXCEPTION
            if scause == 3 {                        //   code 3: a breakpoint
                TRAP_COUNT += 1;
                asm!("csrw sepc, {}", in(reg) sepc + 4);
            }
        }
    }
}
```

(a) Fill in the row for a timer tick, which arrives with `scause = 0x8000_0000_0000_0001`. The first row is done for you. *Hint:* `scause >> 63` is the top bit (bit 63); `scause & 0xff` is the low 8 bits.

| `scause` | top bit | interrupt or exception? | code | what `kerneltrap` does |
|---|---|---|---|---|
| `0x3` | 0 | exception | 3 | breakpoint: `TRAP_COUNT += 1`, `sepc += 4` |
| `0x8000_0000_0000_0001` | | | | |

(b) A student's handler counts the tick but forgets the `csrw sip` line, so SSIP is never cleared. What happens after `sret`, and why?